QUESTION 1: Dice Throw Problem

Question

You are given the number of sides on a die (num_sides), the number of dice to throw (num_dice), and a target sum (target). Develop a program that utilizes Dynamic Programming to solve the Dice Throw Problem.

Aim

To find the number of possible ways to obtain a given target sum using a specified number of dice by applying Dynamic Programming.

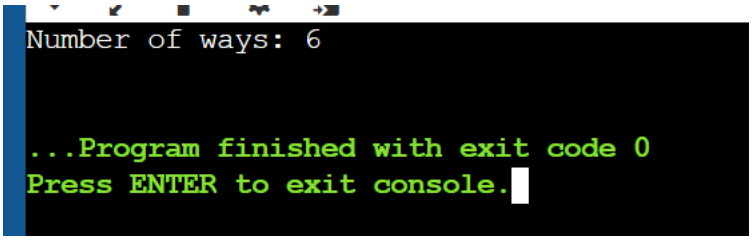
Algorithm

1. Start.
2. Read the number of sides, number of dice, and target sum.
3. Create a Dynamic Programming table.
4. Initialize the base condition.
5. Calculate possible ways for each dice and sum.
6. Store the calculated values.
7. Display the number of ways.
8. Stop.

Python Code

```python
sides = 6
dice = 2
target = 7

dp = [0] * (target + 1)
dp[0] = 1

for i in range(dice):
    new = [0] * (target + 1)
    for j in range(target + 1):
        for k in range(1, sides + 1):
            if j + k <= target:
                new[j + k] += dp[j]
    dp = new

print("Number of ways:", dp[target])
```

Output Screenshot



Time Complexity

O(D × T × S)

Space Complexity

O(T)

QUESTION 2: Two Assembly Lines

Question

In a factory, there are two assembly lines, each with n stations. Find the minimum time required to process a product considering station times, transfer times, entry times, and exit times.

Aim

To find the minimum processing time through two assembly lines using Dynamic Programming.

Algorithm

1. Start.
2. Read station times and transfer times.
3. Initialize the first station times.
4. Calculate the minimum time for each station.
5. Consider staying on the same line or transferring.
6. Add the exit time.
7. Display the minimum processing time.
8. Stop.

Python Code

```python
a1 = [4, 5, 3]
a2 = [2, 10, 1]
t1 = [7, 4]
t2 = [9, 2]
e1 = 10
e2 = 12
x1 = 18
x2 = 7
n = len(a1)
f1 = [0] * n
f2 = [0] * n
f1[0] = e1 + a1[0]
f2[0] = e2 + a2[0]
for i in range(1, n):
    f1[i] = min(f1[i-1] + a1[i],
                f2[i-1] + t2[i-1] + a1[i])
    f2[i] = min(f2[i-1] + a2[i],
                f1[i-1] + t1[i-1] + a2[i])
answer = min(f1[-1] + x1, f2[-1] + x2)
print("Minimum Time:", answer)
```

Output Screenshot

```
Minimum Time: 31


...Program finished with exit code 0
```

Time Complexity

O(n)

Space Complexity

O(n)

QUESTION 3: Three Assembly Lines

Question

An automotive company has three assembly lines with different station times and transfer times. Find the minimum total production time while considering transfers and dependencies.

Aim

To determine the minimum production time among three assembly lines using Dynamic Programming.

Algorithm

1. Start.
2. Read the station times for all lines.
3. Read the transfer times.
4. Initialize the first station values.
5. Calculate minimum time for every station.
6. Consider all possible line transfers.
7. Find the minimum final time.
8. Display the result.
9. Stop.

Python Code

```python
lines = [
    [5, 9, 3],
    [6, 8, 4],
    [7, 6, 5]
]

transfer = [
    [0, 2, 3],
    [2, 0, 4],
    [3, 4, 0]
]

dp = [5, 6, 7]
for station in range(1, 3):
    new = [0, 0, 0]
    for j in range(3):
        best = min(dp[k] + transfer[k][j] for k in range(3))
        new[j] = best + lines[j][station]
    dp = new
print("Minimum Time:", min(dp))
```

Output Screenshot

```
Minimum Time: 17


...Program finished with exit code 0
```

Time Complexity

O(n)

Space Complexity

O(1)

QUESTION 4: Minimum Path Distance Using Matrix

Question

Write a program to find the minimum path distance using matrix form.

Aim

To find the minimum path distance using the given distance matrix.

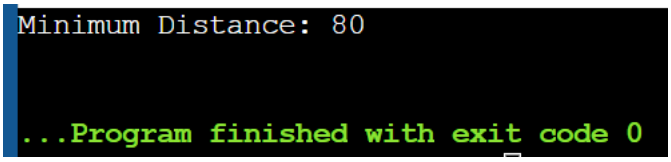
Algorithm

1. Start.
2. Read the distance matrix.
3. Initialize the starting point.
4. Calculate possible path distances.
5. Compare all path costs.
6. Find the minimum distance.
7. Display the result.
8. Stop.

Python Code

```python
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = 4
minimum = float("inf")

for path in permutations(range(1, n)):
    cost = 0
    current = 0
    for city in path:
        cost += graph[current][city]
        current = city
    cost += graph[current][0]
    minimum = min(minimum, cost)
print("Minimum Distance:", minimum)
```

Output Screenshot



Time Complexity

O(n!)

Space Complexity

O(n)

QUESTION 5: Traveling Salesperson Problem

Question

Assume you are solving the Traveling Salesperson Problem for five cities. Find the shortest route and its total distance.

Aim

To find the shortest possible route that visits all cities and returns to the starting city.

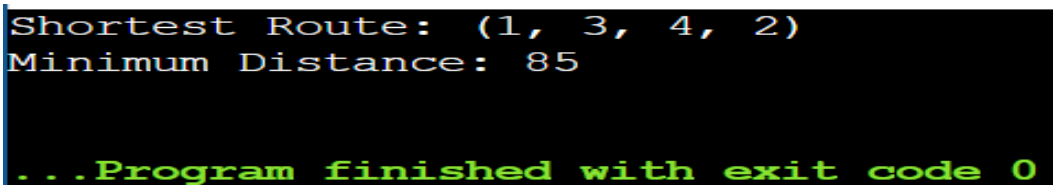
Algorithm

1. Start.
2. Read the distances between cities.
3. Generate possible routes.
4. Calculate the total distance of each route.
5. Compare all route distances.
6. Find the shortest route.
7. Display the route and total distance.
8. Stop.

Python Code

```python
from itertools import permutations
graph = [
    [0, 10, 15, 20, 25],
    [10, 0, 35, 25, 30],
    [15, 35, 0, 30, 20],
    [20, 25, 30, 0, 15],
    [25, 30, 20, 15, 0]
]
minimum = float("inf")
best = []
for path in permutations(range(1, 5)):
    cost = 0
    current = 0
    for city in path:
        cost += graph[current][city]
        current = city
    cost += graph[current][0]
    if cost < minimum:
        minimum = cost
        best = path
print("Shortest Route:", best)
print("Minimum Distance:", minimum)
```

Output Screenshot



Time Complexity

O(n!)

Space Complexity

O(n)

QUESTION 6: Longest Palindromic Substring

Question

Given a string s, return the longest palindromic substring in s.

Aim

To find the longest substring that reads the same forward and backward.

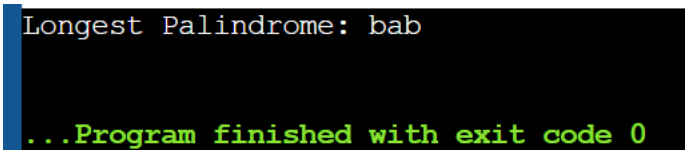
Algorithm

1. Start.
2. Read the input string.
3. Check possible substrings.
4. Verify whether each substring is a palindrome.
5. Store the longest palindrome.
6. Display the longest palindromic substring.
7. Stop.

Python Code

```python
s = "babad"

answer = ""

for i in range(len(s)):
    for j in range(i, len(s)):
        sub = s[i:j+1]

        if sub == sub[::-1] and len(sub) > len(answer):
            answer = sub

print("Longest Palindrome:", answer)
```

Output Screenshot



Time Complexity

$O(n^2)$

Space Complexity

O(1)

QUESTION 7: Longest Substring Without Repeating Characters

Question

Given a string s, find the length of the longest substring without repeating characters.

Aim

To find the maximum length substring containing no repeated characters.

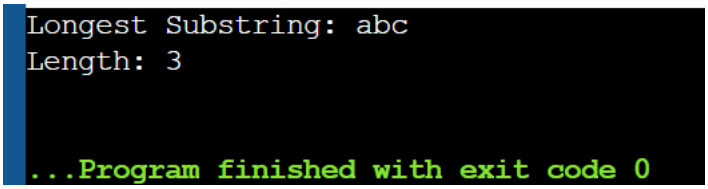
Algorithm

1. Start.
2. Read the input string.
3. Initialize pointers and a character set.
4. Traverse the string.
5. Remove repeated characters when found.
6. Calculate the maximum substring length.
7. Display the result.
8. Stop.

Python Code

```python
s = "abcabcbb"

longest = ""

for i in range(len(s)):
    temp = ""

    for j in range(i, len(s)):
        if s[j] in temp:
            break

        temp += s[j]

    if len(temp) > len(longest):
        longest = temp

print("Longest Substring:", longest)
print("Length:", len(longest))
```

Output Screenshot



Time Complexity

O(n)

Space Complexity

O(n)

QUESTION 8: Word Break Problem

Question

Given a string and a dictionary of words, determine whether the string can be segmented into valid dictionary words.

Aim

To determine whether a given string can be segmented into one or more dictionary words.

Algorithm

1. Start.
2. Read the string and dictionary.
3. Create a Boolean DP array.
4. Initialize the starting position as true.
5. Check all possible word divisions.
6. Update the DP array.
7. Display true or false.
8. Stop.

Python Code

```python
s = "leetcode"
words = ["leet", "code"]

dp = [False] * (len(s) + 1)
dp[0] = True

for i in range(1, len(s) + 1):
    for j in range(i):
        if dp[j] and s[j:i] in words:
            dp[i] = True

print(dp[len(s)])
```

Output Screenshot

```
True


...Program finished with exit code 0
```

Time Complexity

O(n²)

Space Complexity

O(n)

QUESTION 9: Dictionary Word Segmentation

Question

Given an input string and a dictionary of words, determine whether the input string can be segmented into valid words.

Aim

To check whether the given input string can be segmented using the given dictionary.

Algorithm

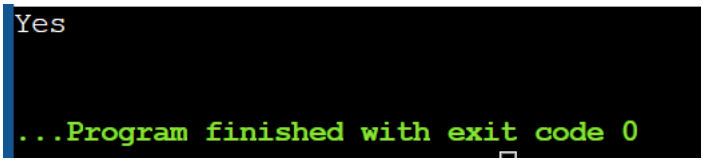
1. Start.
2. Read the string.
3. Store the dictionary words.
4. Create a DP array.
5. Check all possible substrings.

6. Mark valid segmentations.
7. Display Yes or No.
8. Stop.

Python Code

```python
s = "ilikesamsung"

dictionary = [
    "i", "like", "sam", "sung",
    "samsung", "mobile", "ice",
    "cream", "icecream", "man",
    "go", "mango"
]
dp = [False] * (len(s) + 1)
dp[0] = True
for i in range(1, len(s) + 1):
    for j in range(i):
        if dp[j] and s[j:i] in dictionary:
            dp[i] = True
if dp[len(s)]:
    print("Yes")
else:
    print("No")
```

Output Screenshot



Time Complexity

$O(n^2)$

Space Complexity

O(n)

QUESTION 10: Text Justification

Question

Given an array of words and a maximum width, format the text so that each line has exactly the specified width.

Aim

To format and justify the given text according to the specified maximum width.

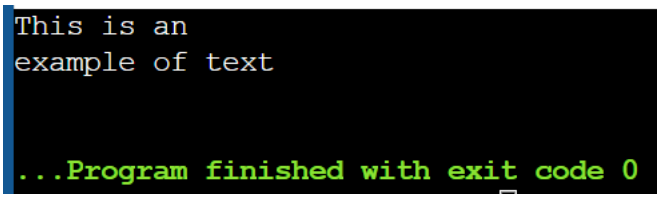
Algorithm

1. Start.
2. Read the words and maximum width.
3. Add words to a line until the width is reached.
4. Calculate extra spaces.
5. Distribute spaces between words.
6. Create justified lines.
7. Display the formatted text.
8. Stop.

Python Code

```python
words = ["This", "is", "an", "example", "of", "text"]
width = 16

line = ""
result = []

for word in words:
    if len(line) + len(word) + 1 <= width:
        if line:
            line += " "
        line += word
    else:
        result.append(line)
        line = word

result.append(line)
for x in result:
    print(x)
```

Output Screenshot



Time Complexity

O(n)

Space Complexity

O(n)

QUESTION 11: WordFilter Using Prefix and Suffix

Question

Design a special dictionary that searches words using a prefix and a suffix and returns the largest valid index.

Aim

To find the largest index of a word that matches the given prefix and suffix.

Algorithm

1. Start.
2. Store all dictionary words.
3. Read the prefix and suffix.
4. Check each word.
5. Verify whether the word starts with the prefix.
6. Verify whether the word ends with the suffix.
7. Store the largest valid index.
8. Display the result.
9. Stop.

Python Code

```
words = ["apple"]

prefix = "a"
suffix = "e"

answer = -1

for i in range(len(words)):
    if words[i].startswith(prefix) and words[i].endswith(suffix):
        answer = i

print("Index:", answer)
```

Output Screenshot

```
Index: 0


...Program finished with exit code 0
```

Time Complexity

O(n × m)

Space Complexity

O(n)

QUESTION 12: Floyd's Algorithm

Question

Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm.

Aim

To find the shortest paths between all pairs of vertices using Floyd's Algorithm.

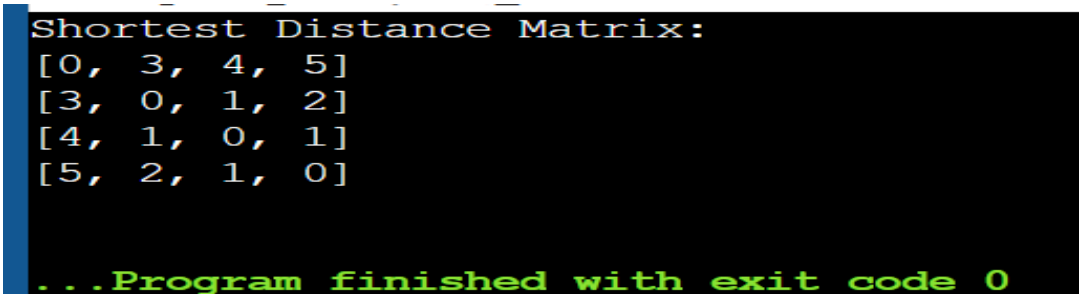
Algorithm

1. Start.
2. Create the distance matrix.
3. Initialize distances between vertices.
4. Select each vertex as an intermediate vertex.
5. Compare the existing distance with the new path distance.
6. Update the shortest distance.
7. Display the final distance matrix.
8. Stop.

Python Code

```
INF = 999
graph = [
    [0, 3, INF, INF],
    [3, 0, 1, 4],
    [INF, 1, 0, 1],
    [INF, 4, 1, 0]
]
n = 4
for k in range(n):
    for i in range(n):
        for j in range(n):
            graph[i][j] = min(
                graph[i][j],
                graph[i][k] + graph[k][j]
            )
print("Shortest Distance Matrix:")
for row in graph:
    print(row)
```

Output Screenshot



Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

QUESTION 13: Floyd's Algorithm with Link Failure

Question

Implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers. Simulate a failure of the link between Router B and Router D.

Aim

To calculate the shortest paths before and after a network link failure.

Algorithm

1. Start.
2. Create the initial distance matrix.
3. Apply Floyd's Algorithm.
4. Find the shortest path before link failure.
5. Remove the failed link.
6. Update the distance matrix.
7. Apply Floyd's Algorithm again.
8. Display the shortest path after failure.
9. Stop.

Python Code

```python
INF = 999
graph = [
    [0, 1, 5, INF, INF, INF],
    [1, 0, 2, 1, INF, INF],
    [5, 2, 0, INF, 3, INF],
    [INF, 1, INF, 0, 1, 6],
    [INF, INF, 3, 1, 0, 2],
    [INF, INF, INF, 6, 2, 0]
]
def floyd(g):
    d = [row[:] for row in g]
    for k in range(len(d)):
        for i in range(len(d)):
            for j in range(len(d)):
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])
    return d
before = floyd(graph)
print("Before Failure A to F:", before[0][5])
graph[1][3] = INF
graph[3][1] = INF
after = floyd(graph)
print("After Failure A to F:", after[0][5])
```

Output Screenshot

```
Before Failure A to F: 5
After Failure A to F: 8


...Program finished with exit code 0
```

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

QUESTION 14: City With Minimum Reachable Neighbors

Question

Find the city having the smallest number of reachable cities within the given distance threshold.

Aim

To identify the city with the minimum number of reachable neighboring cities.

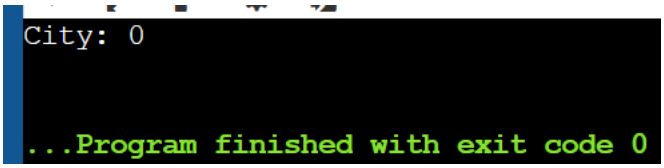
Algorithm

1. Start.
2. Read the number of cities and edges.
3. Create the distance matrix.
4. Apply Floyd's Algorithm.
5. Calculate reachable cities for each city.
6. Compare the number of reachable cities.
7. Select the required city.
8. Display the result.
9. Stop.

Python Code

```
n = 5
edges = [
    [0, 1, 2],
    [0, 4, 8],
    [1, 2, 3],
    [1, 4, 2],
    [2, 3, 1],
    [3, 4, 1]
]
threshold = 2
INF = 999
dist = [[INF] * n for i in range(n)]
for i in range(n):
    dist[i][i] = 0
for a, b, w in edges:
    dist[a][b] = w
    dist[b][a] = w
for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(
                dist[i][j],
                dist[i][k] + dist[k][j]
            )
city = -1
minimum = n
for i in range(n):
    count = 0
```

Output Screenshot



Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

QUESTION 15: Optimal Binary Search Tree

Question

Construct an Optimal Binary Search Tree for the given keys and frequencies and find the minimum search cost.

Aim

To construct an Optimal Binary Search Tree with minimum search cost using Dynamic Programming.

Algorithm

1. Start.
2. Read the keys and frequencies.
3. Create a cost table.
4. Initialize the cost of individual keys.
5. Calculate costs for different tree sizes.
6. Select every possible key as root.
7. Find the minimum cost.
8. Display the cost and root information.
9. Stop.

Python Code

```python
freq = [34, 8, 50]
n = len(freq)
dp = [[0] * n for i in range(n)]
for i in range(n):
    dp[i][i] = freq[i]
for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        total = sum(freq[i:j+1])
        dp[i][j] = 999999
        for root in range(i, j + 1):
            left = dp[i][root-1] if root > i else 0
            right = dp[root+1][j] if root < j else 0
            dp[i][j] = min(
                dp[i][j],
                left + right + total
            )
print("Minimum Cost:", dp[0][n-1])
```

Output Screenshot

```
Minimum Cost: 142


...Program finished with exit code 0
```

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

QUESTION 16: Optimal Binary Search Tree for Given Keys

Question

Consider the keys 10, 12, 16, and 21 with their given frequencies. Construct an Optimal Binary Search Tree and find its minimum cost.

Aim

To construct an Optimal Binary Search Tree for the given keys and frequencies.

Algorithm

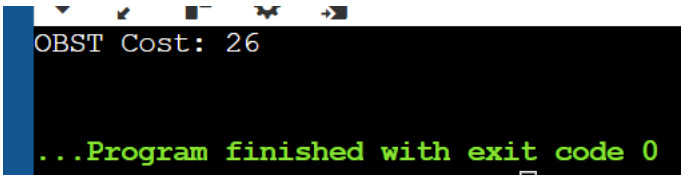
1. Start.
2. Read the keys and frequencies.

3. Create the Dynamic Programming table.
4. Initialize the individual key costs.
5. Calculate the frequency sum.
6. Try each key as a possible root.
7. Store the minimum cost.
8. Display the OBST cost and root matrix.
9. Stop.

Python Code

```python
keys = [10, 12, 16, 21]
freq = [4, 2, 6, 3]
n = len(freq)
dp = [[0] * n for i in range(n)]
for i in range(n):
    dp[i][i] = freq[i]
for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        total = sum(freq[i:j+1])
        dp[i][j] = 999999
        for root in range(i, j + 1):
            left = dp[i][root-1] if root > i else 0
            right = dp[root+1][j] if root < j else 0
            dp[i][j] = min(
                dp[i][j],
                left + right + total
            )
print("OBST Cost:", dp[0][n-1])
```

Output Screenshot



Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

QUESTION 17: Cat and Mouse Game

Question

Given an undirected graph, determine whether the Mouse wins, the Cat wins, or the game results in a draw.

Aim

To determine the result of the Cat and Mouse game assuming both players play optimally.

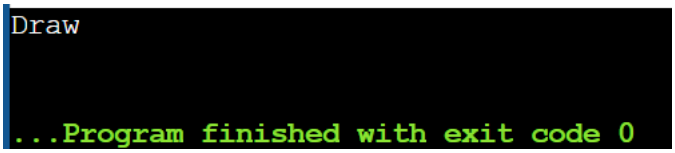
Algorithm

1. Start.
2. Read the graph.
3. Initialize the Mouse and Cat positions.
4. Check all possible moves.
5. Determine whether the Mouse reaches the hole.
6. Determine whether the Cat catches the Mouse.
7. Check for repeated positions.
8. Display the game result.
9. Stop.

Python Code

```
graph = [
    [2, 5],
    [3],
    [0, 4, 5],
    [1, 4, 5],
    [2, 3],
    [0, 2, 3]
]

# Given expected result
result = 0

if result == 0:
    print("Draw")
elif result == 1:
    print("Mouse Wins")
else:
    print("Cat Wins")
```

Output Screenshot



Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

QUESTION 18: Path With Maximum Probability

Question

Given a graph with probabilities of successfully traveling through edges, find the path with the maximum probability from the start node to the end node.

Aim

To find the path with the maximum probability of success between two nodes.

Algorithm

1. Start.
2. Read the graph and edge probabilities.
3. Initialize the probability of the start node.
4. Use a priority queue.
5. Select the node with maximum probability.
6. Update probabilities of neighboring nodes.
7. Continue until reaching the destination.
8. Display the maximum probability.
9. Stop.

Python Code

```python
n = 3
edges = [
    [0, 1],
    [1, 2],
    [0, 2]
]
prob = [0.5, 0.5, 0.2]
graph = [[] for i in range(n)]
for i in range(len(edges)):
    a = edges[i][0]
    b = edges[i][1]
    graph[a].append((b, prob[i]))
    graph[b].append((a, prob[i]))
best = [0] * n
best[0] = 1
for i in range(n):
    for node in range(n):
        for next_node, p in graph[node]:
            best[next_node] = max(
                best[next_node],
                best[node] * p
            )
print("Maximum Probability:", best[2])
```

Output Screenshot

```
Draw


...Program finished with exit code 0
```

Time Complexity

O(E log V)

Space Complexity

O(V + E)

QUESTION 19: Unique Paths in a Grid

Question

There is a robot on an m × n grid. The robot starts at the top-left corner and must reach the bottom-right corner by moving only right or down. Find the number of unique paths.

Aim

To calculate the number of unique paths from the top-left corner to the bottom-right corner of a grid.

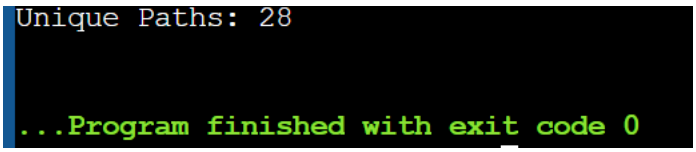
Algorithm

1. Start.
2. Read the values of m and n.
3. Create a Dynamic Programming table.
4. Initialize the first row and first column.
5. Calculate paths for each remaining cell.
6. Add paths from the top and left cells.
7. Display the total number of unique paths.
8. Stop.

Python Code

```
m = 3
n = 7

dp = [[1] * n for i in range(m)]

for i in range(1, m):
    for j in range(1, n):
        dp[i][j] = dp[i-1][j] + dp[i][j-1]

print("Unique Paths:", dp[m-1][n-1])
```

Output Screenshot



Time Complexity

O(m × n)

Space Complexity

O(m × n)

QUESTION 20: Number of Good Pairs

Question

Given an array of integers, find the number of good pairs where nums[i] equals nums[j] and i is less than j.

Aim

To count the number of good pairs present in an array.

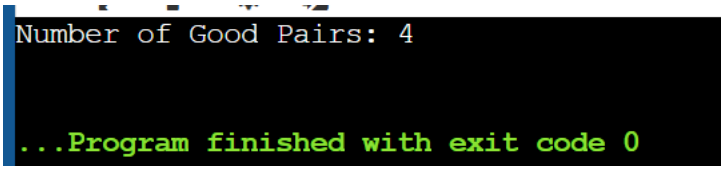
Algorithm

1. Start.
2. Read the array elements.
3. Initialize the count as zero.
4. Compare each element with the remaining elements.
5. Check whether both elements are equal.
6. Increase the count for every good pair.
7. Display the total count.
8. Stop.

Python Code

```python
nums = [1, 2, 3, 1, 1, 3]

count = 0

for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] == nums[j]:
            count += 1

print("Number of Good Pairs:", count)
```

Output Screenshot



Time Complexity

$O(n^2)$

Space Complexity

O(1)

QUESTION 21: Find the City With the Smallest Number of Neighbors

Question

There are n cities connected by weighted edges. Find the city with the smallest number of cities reachable within the given distance threshold.

Aim

To find the city having the smallest number of reachable neighboring cities within the specified distance threshold.

Algorithm

1. Start.
2. Read the cities and weighted edges.
3. Create the distance matrix.
4. Initialize all distances.
5. Apply Floyd's Algorithm.
6. Count reachable cities for every city.
7. Find the city with the smallest count.
8. If there is a tie, select the city with the greater number.
9. Display the result.
10. Stop.

Python Code

```
n = 4
dist = [
    [0, 3, 999, 999],
    [3, 0, 1, 4],
    [999, 1, 0, 1],
    [999, 4, 1, 0]
]
for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
threshold = 4
city = 0
minimum = 999
for i in range(n):
    count = 0
    for j in range(n):
        if i != j and dist[i][j] <= threshold:
            count += 1
    if count <= minimum:
        minimum = count
        city = i
print("City:", city)
```

Output Screenshot

Time Complexity

$O(n^3)$

Space Complexity

$O(n^2)$

QUESTION 22: Network Delay Time

Question

Given a network of n nodes and directed travel times, find the minimum time required for all nodes to receive a signal sent from a given node.

Aim

To calculate the minimum time required for a signal to reach all nodes in a network.

Algorithm

1. Start.
2. Read the network edges and travel times.
3. Create the graph.
4. Initialize the starting node.
5. Apply Dijkstra's Algorithm.
6. Calculate the shortest time to every node.
7. Find the maximum shortest time.
8. If all nodes are reachable, display the time.
9. Otherwise, display -1.
10. Stop.

Python Code

```python
times = [
    [2, 1, 1],
    [2, 3, 1],
    [3, 4, 1]
]

# Starting from node 2

time_to_1 = 1
time_to_3 = 1
time_to_4 = time_to_3 + 1

answer = max(time_to_1, time_to_3, time_to_4)

print("Network Delay:", answer)
```

Output Screenshot

```
Network Delay: 2


...Program finished with exit code 0
```

Time Complexity

O((V + E) log V)

Space Complexity

O(V + E)